

Skillet 0.0.5 — consolidated product brief

Reviewed 12 September 2026. **This is the next product/design target, not a shipped application release.** It incorporates Paul's requested changes, a README/code audit, and a comparison with five other skill managers.

Current version and delivery status

Artifact	Verified state
Application source	Bun workspace; root package declares 0.1.0. No version change or new application release was made for this brief.
Latest Skillet Paper file	<u>Skillet — 0.0.4</u> , five current desktop boards: inventory, prompt, sharing, New Folder, interaction states.
Paper history	Earlier full flows remain on page 2-0. Current page/artboard labels still contain v4; the file title uses 0.0.4.
Approved common components	<u>Components Paper library</u> ; local tokens, HTML templates and previews in Components/patterns/shared-ui. These are not yet a working React component package. Its local manifest still says 4.0.0.
This document	0.0.5: consolidated requirements and implementation handoff.

There is currently **no runnable Skillet build containing all the requested design and sharing changes**. The code still has green accents, “View prompt,” “Share & transfer,” “Copy prompt,” and “Add project folder.” The common library itself confirms that its themes have not recolored Skillet. The new requirements below must not be described as already implemented.

Product definition

Skillet is a local desktop-oriented control panel for AI coding skills: find every copy, understand which agent can use it, edit its instructions and invocation settings, and move or share it with a reviewable result.

Its strongest existing foundation is the combination of project/worktree discovery, a visibility matrix, duplicate comparison, canonical hub storage, and reversible filesystem operations. Preserve that depth while making everyday actions compact and easy to find.

Everything requested for 0.0.5

Requirement	Current state	Acceptance for the target
Yellow Skillet theme	Approved direction; app and current Skillet boards remain green	Apply the shared 1emon semantic tokens across all main screens, controls and states.
Light mode only	App already declares light appearance	Keep every theme light, including overlays and loading/error states; no automatic dark variant.
Subtle background gradients	Present in common design templates	Gradients only on page/card/modal backgrounds. Buttons have solid pastel fills in default, hover and pressed states.
Compact clickable cards	Paper refinement exists; app still uses a View button	Clicking/keyboard-activating the card opens the skill. Independent share icon in the top-right corner. No nested interactive controls.
+ New Folder	Paper refinement exists	Use this label for adding an existing project directory to the scan; do not imply it creates a filesystem folder.
Contextual icons	Paper refinement exists	Copy icon beside the prompt, corner share icon, close icon in modals. Accessible names, tooltips and visible focus; retain labels for unfamiliar or consequential actions.
Reusable chips	Paper masters exist	Reuse agent, scope and category chips. Clear selected/filter states. Avoid one-off badges with inconsistent padding.

Requirement	Current state	Acceptance for the target
Compact rounded modals	Paper refinement exists	Consistent header/body/footer alignment, brief copy, grouped fields, one main action, focus restoration and recoverable errors.
Clean statuses	Approved common components	Small semantic icon plus readable neutral label; no full-width outlined status strips.
Shared component library outside products	Tokens/templates exist in Components	Build reusable working components in Components, then copy approved components into Skillet with origin/version recorded. Skillet must run independently of the Desktop library checkout.
Five proper themes	Common library has lemon, rose, coral, sky, lavender	Reuse shared geometry and semantic roles; only palette/background effects vary. Skillet defaults to lemon.
Global skill at top level	Make global action exists	Global skills have a first-class top-level view. Promotion previews the new hub location and agent links, preserves project access where intended, then updates the visible scope.
Change when invoked	Editor exists, with compatibility issues	Edit the description and supported agent-specific invocation policy. Explain which files and linked agents are affected before saving.
Reliable sharing outside this machine	Missing	Export a complete portable skill ZIP; import it with destination, conflict and compatibility review.
Copy to another agent	Raw local copy/link exists	Choose agent and global/project scope; distinguish an independent copy from a shared link; report unsupported settings and prerequisites.
Semver-style design versions	File title partially normalized	Use 0.0.x for design iterations, preserve prior pages and document version mappings. Do not silently equate design, app and skill-package versions.
Desktop focus	Accepted	Complete desktop Skillet flows first; retain existing responsive functionality. Mobile redesign is deferred. HeartTap remains a separate project.

Four main workflows

1. Make global

Open skill → Make global → see canonical hub destination and agents that will gain access → review filesystem changes → apply → return to the Global view with feedback and undo.

The hub directory alone does not guarantee that every agent discovers the skill. The existing action moves to the hub and retains the source link; the target must explicitly account for required global agent links. Existing names must trigger a conflict choice, never a silent overwrite.

2. Change invocation

Open skill → Invocation → choose agent → edit “Use when...” description and supported options → preview affected files → save.

Keep “automatic selection,” “explicit invocation only,” and “disabled” distinct. The description influences model selection; matching a description is not a deterministic guarantee that a skill will run.

Codex documents per-skill agents/openai.yaml policy for implicit invocation and path-based [[skills.config]] entries for disabling a local skill. Claude Code uses its own frontmatter and settings. These need separate adapters. [Codex skills documentation](#), [Claude Code skills documentation](#).

3. Share a portable file

Share icon → Export ZIP → preview included files, version and compatibility findings → download skill-name-0.0.1.zip → attach the file in Slack. Recipient downloads it → imports into Skillet → selects agent/scope → reviews → installs.

Recommended package layout:

```
skill-name-0.0.1.zip
└─ skill-name/
  │ SKILL.md
  │ scripts/      # when referenced
  │ references/   # when referenced
  │ assets/       # when referenced
  └─ agents/      # applicable vendor metadata
```

Use a standard ZIP containing the actual skill folder, not an absolute local link or a copied prompt. Preserve relative references, required resources and licenses. Include a content manifest/checksum for Skillet's import review. Exclude machine credentials, caches and unrelated files; flag external symlink targets rather than silently packaging them. Archive extraction must reject path traversal and unsafe links and impose size/file-count limits.

Claude documents ZIP packaging with one skill-folder root. Slack supports file uploads up to 1 GB, subject to workspace/Connect restrictions and its file scanning; successful upload is not a guarantee that another agent can run the contents. A ZIP is the recommended transport, not a special Slack skill format. No Slack sending is implemented or performed here. [Claude packaging guide](#), [Slack file upload documentation](#).

4. Copy to another agent

Share icon → Copy to agent → choose Claude Code, Codex or another target → select global/project → review compatibility and destination → apply → inspect resulting availability.

The shared SKILL.md structure is portable, but vendor metadata, tool names, scripts, absolute paths and required MCP servers may not be. A copied folder is not automatically a converted workflow. Preserve the original; preview any adapter changes. Distinguish Claude Code from Claude web uploads, which accept a narrower frontmatter schema. [Agent Skills specification](#), [Claude Code cross-product compatibility](#).

Existing capabilities to preserve

Code paths exist for global/project/plugin scanning, configured project roots and worktrees, installed/custom agent detection, search/filtering, Markdown/frontmatter editing, instance grouping, diffs, visibility conditions, local hub adoption, linking, copying, moving, renaming, trash/restore, history/undo, filesystem previews, rollback and live refresh.

These are implementation observations, not a claim that all runtime behavior was retested. Plugin-managed originals remain read-only. Local linked sharing, user-to-user distribution, and agent-format adaptation must have separate wording.

What other managers have that Skillet lacks

Compared against current maintainer READMEs on the review date. “Missing” means absent from Skillet's inspected UI/API/code paths; competitor capabilities are documented claims, not independently executed tests. Roadmap-only features are excluded.

Gap	Other manager with documented support	Skillet today	Recommendation
Search/discover and install from Git repositories	Vercel skills , SkillDeck	Finds local installations; no remote installer	High: paste repository URL first, then add a discovery browser.
Check for upstream changes and update	Vercel skills , SkillDeck	Local divergence comparison only	High: retain source identity/revision, preview remote-vs-local changes, preserve local edits.
Download/archive installation	Vercel skills accepts skill documents and ZIP/tar download sources	No archive import/export route	High: implement the user's ZIP round-trip first.
Create a skill from a template	Skill Manager — abubakarsiddik31	Edits existing skills only	Medium: compact New Skill flow with description, scope and agent.
Scan imported skills for risky content	Skill Manager — abubakarsiddik31 documents SkillSpector analysis	Schema validation and filesystem guards; no content scanner	High alongside remote imports; findings are signals, not proof of safety.

Gap	Other manager with documented support	Skillet today	Recommendation
Suggest projects and pin frequent ones	Skill Manager — abubakarsiddik31	Manual project roots and search	Medium: pinned projects first; make history/editor-recents discovery optional.
Native installers	Skills Manager — jiweiyeah provides macOS, Windows and Linux packages	Bun development workflow	Medium: Linux package/AppImage is especially relevant to Paul's setup.
CLI, diagnostics and repair	Skills Manager — jiweiyeah documents <code>skm doctor</code> , <code>fix</code> and a companion skill	HTTP API, no equivalent packaged CLI	Medium: reuse existing preview/apply operations and machine-readable output.
Agent-driven management	SkillDeck ships a management skill	No packaged Skillet companion skill or MCP service found	Medium: expose existing operations through a narrow adapter with previews.
Windows link fallbacks	Skills Manager — jiweiyeah documents junction and tracked-copy fallbacks	Symlink/copy operations; no equivalent fallback workflow found	Later, when Windows becomes a delivery target.
Translation/localization	SkillDeck , Skills Manager — jiweiyeah	No equivalent interface found	Later; limited value for the immediate workflow.
Private publishing with package versions and team permissions	SkillHub — iFlytek	Local single-user hub and journal	Later: useful for a team product, excessive for basic Slack sharing.

There is substantial overlap: central libraries, symlinks, editors and per-agent controls are not unique to Skillet. Its useful positioning is **explainable availability plus reversible local changes**. Avoid the README's unsupported universal claim that no competitor explains visibility.

README and implementation audit

Finding	Evidence	Required correction
"No code splitting" is stale	<code>apps/web/src/app/App.tsx</code> lazy-loads <code>SkillDetailPage</code>	Describe the current split; measure before quoting a bundle size.
Skills list described as rows	<code>components/SkillCards.tsx</code> renders articles/cards	Update the tour to match the actual interface.
Codex disabling schema mismatch	<code>hub/trigger.ts</code> writes a skill-name-keyed object; current official docs use path-based <code>[[skills.config]]</code>	Fix reader and writer together; test actual Codex-recognized settings before claiming support.
Claude trigger assumptions leak across agents	<code>visibility/visibility.ts</code> applies frontmatter flags and path conditions generally	Gate rules by verified agent capability. Show unknown/unsupported behavior honestly.
Codex implicit policy is missing	No <code>allow_implicit_invocation</code> handling in the inspected app; official docs and installed skill-creator reference describe it	Read/write the selected skill's <code>agents/openai.yaml</code> with preview and undo.
Invalid settings may be discarded during trigger edits	<code>hub/trigger.ts</code> catches JSON/TOML parse errors and substitutes an empty object	Stop with a repairable parse error instead of replacing unrelated configuration.
"Copies differ by content" overstates detection	<code>scan/walk.ts</code> hashes SKILL.md contents plus filenames; <code>hub/diff.ts</code> compares the skill document	Supporting-file content changes can be missed. Hash all packaged files and offer file-level diffs.
Safety language needs narrower scope	README says every mutation has the same preview/undo guarantees; route behaviors differ	Document which operations are journaled, previewed and undoable; distinguish config saves and permanent purge.
Version identifiers disagree	App 0.1.0, Paper file 0.0.4, internal v4, common manifest 4.0.0	Maintain an explicit artifact/version table and normalize labels deliberately.

The README's high-level explanation of the local hub and visibility problem remains useful. Its getting-started clone URL is still a placeholder. It should link to this brief until the requested changes are implemented, rather than presenting target behavior as shipped.

Recommended build order

1. **Correctness first:** vendor-specific visibility/invoke, Codex config schema, refusal to overwrite malformed settings, whole-folder hashing. Add isolated mutation fixtures.
2. **Finish the approved interface:** shared working components, lemon theme, clickable cards, icon affordances, compact dialogs and Global navigation; reconcile all affected Paper flows.
3. **Portable sharing:** ZIP export/import, resource completeness, conflict handling, compatibility report, and copy-to-agent workflow.
4. **Acquire and maintain skills:** URL install, provenance, update preview and New Skill templates. Add import scanning with remote acquisition.
5. **Convenience and distribution:** pinned projects, desktop packaging, CLI/companion skill; consider private registry and localization only when needed.

Release acceptance: an isolated skill with a script and reference must survive export → import on a different root → target-agent installation; trigger settings must be recognized by the actual supported agent; edits to supporting files must trigger divergence; failures must preserve unrelated settings and allow recovery. Then run `bun run quality` and `bun run test:browser`, including keyboard/focus and main desktop flows.

Verification performed for this brief

Read the complete README, repository guidance, current UI/action and API routes, trigger/visibility/linking/hash/diff implementations, existing sharing browser test, common library usage notes, and live Paper metadata. Compared the three prior-art repositories already named in the README plus SkillDeck and iFlytek SkillHub. Checked official Codex, Claude, Agent Skills and Slack documentation.

`bun run test:unit`: **1 test file, 3 tests passed**. These are parsing tests; they do not verify the identified trigger or transfer issues. No end-to-end run, broad frontend change, source release, agent-config mutation or Slack message was performed for this report.